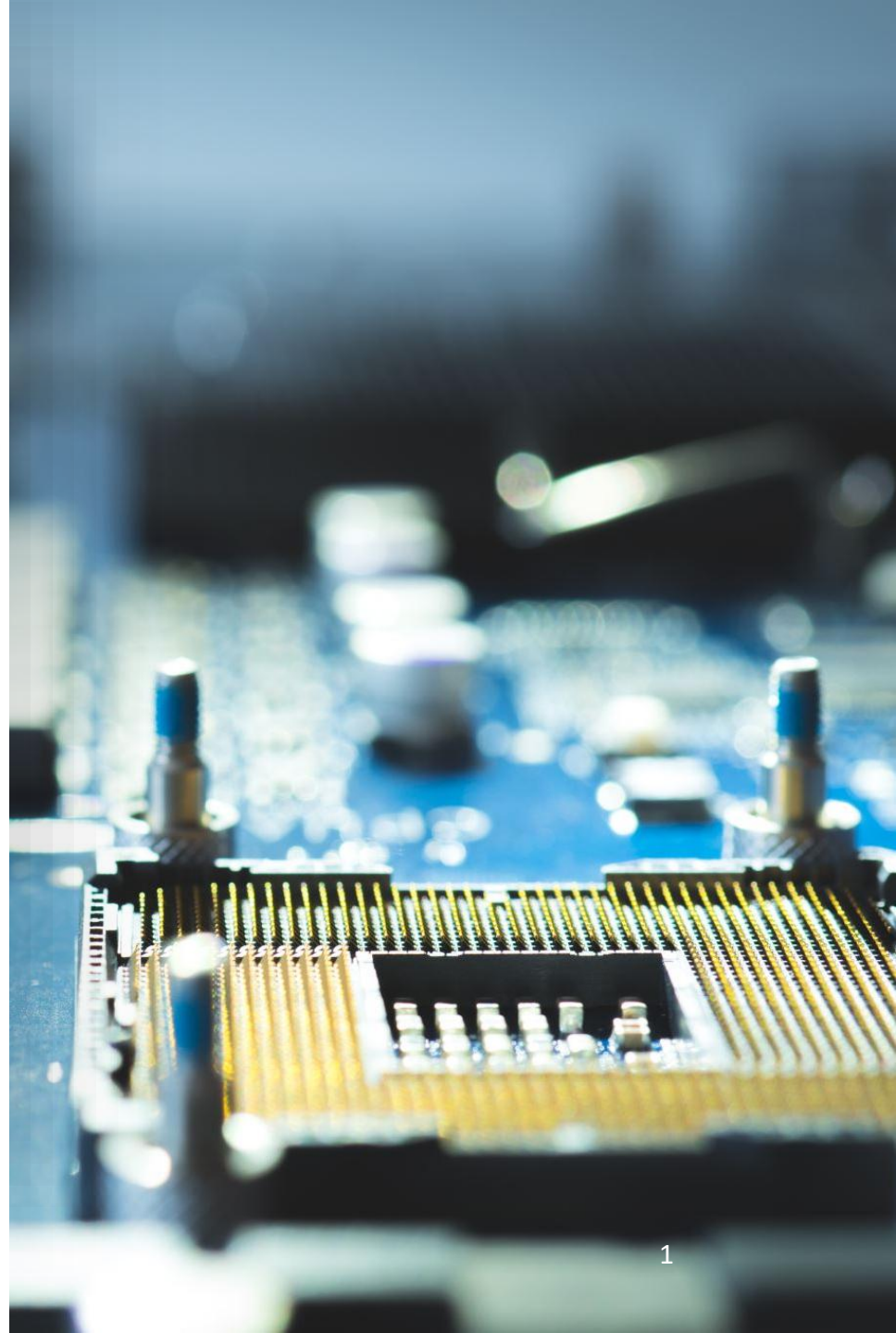


Programming II

Methods (Functions)



Text Books

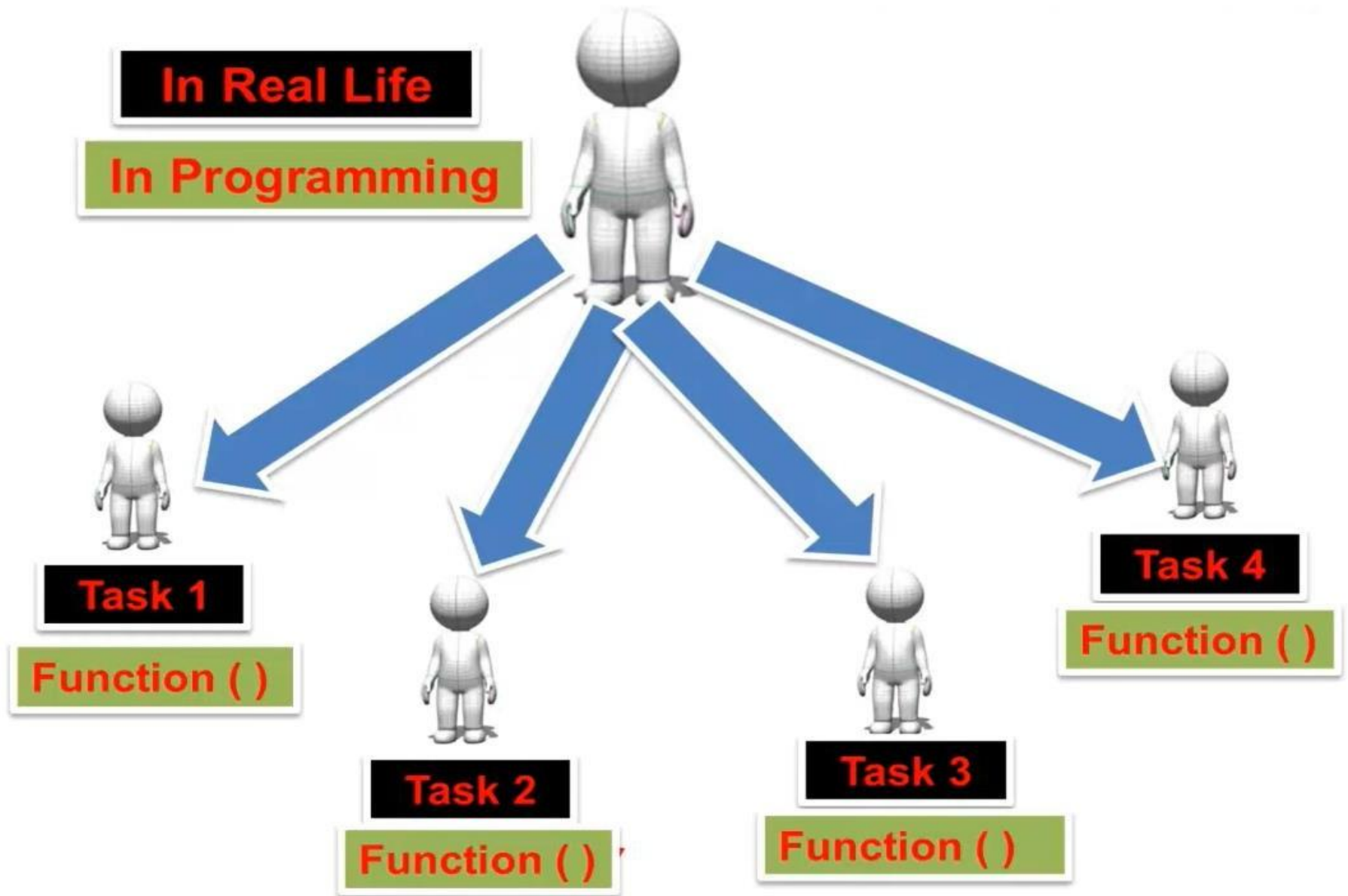
Building Java Programs: A Back to Basics Approach
(4th Edition) Stuart Reges & Marty Stepp;

Y. Daniel Liang, 2020. Java How to Program:
Comprehensive Version. 12th ed. Pearson.

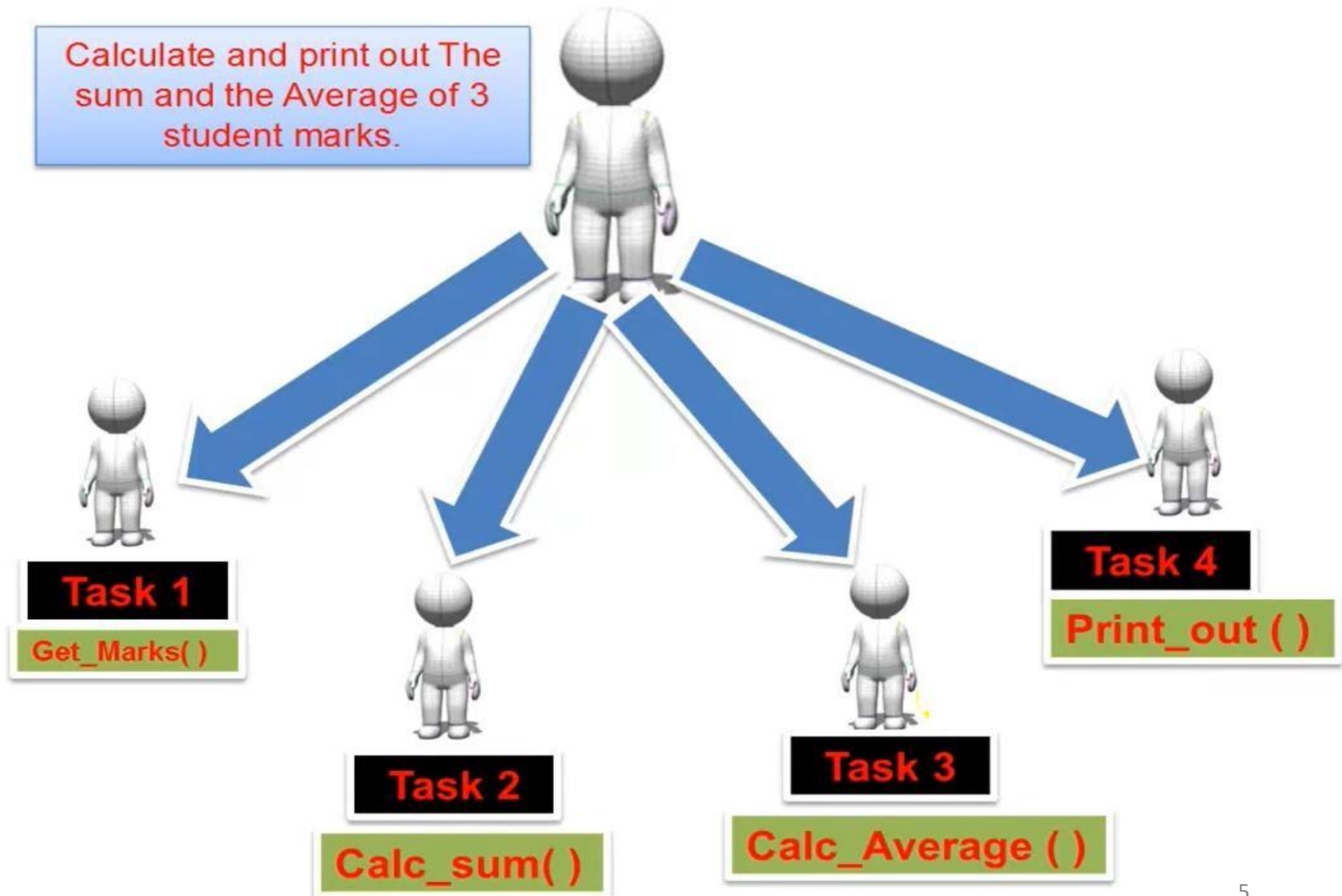
Method (Function):

- Most computer programs that solve real-world problems include hundreds even thousands of lines.
- Experience has shown that the best way to develop and maintain a large program is to construct it from smaller pieces or modules, each of which is more manageable than the original program.
- Method (Function) is a self-contained block of statements that perform a specific task.
- Using a function is something like hiring a person to do a specific job for you.

Method:



Method:



Function Types:

1 Built in Functions:

- **Example: Math functions.**

```
double x = 25;  
System.out.println (Math.sqrt(x));
```

2- User Defined Functions.

2- User Defined Functions:

Syntax:

```
return_value_type function_name ( parameter list)
{
    .....
    body of the function
    .....
    return (...);
}
```

Parameter List:

- It is a list of data values supplied from the calling program as input to the function.
- It is optional.

return-value-type:

- It is the data type for the returned value from the function to the calling program.
- It is Mandatory, if no returned value expected, we use keyword void.

```
void function_name ( parameter list)  
    {  
        .....  
        body of the function  
        .....  
    }
```

- In void functions do not use a **return** statement to return a value.

2- Calling a Function (From the main ())

Syntax:

Function name (Actual Parameter-list)

```
public static void main (string [ ] args )
```

```
{
```

```
... ..  
... ..
```

```
functionName();
```

```
... ..  
... ..
```

```
}
```

```
void functionName()
```

```
{
```

```
... ..  
... ..
```

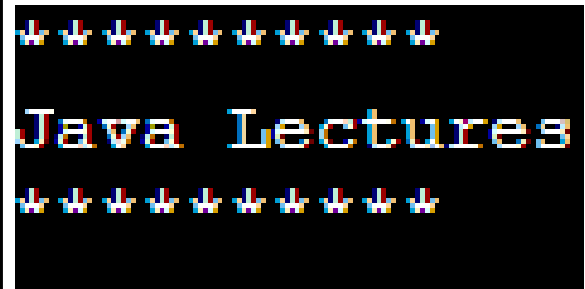
```
}
```

Example

Write a program using functions to write a statement between two-star lines.

```
public class Main
{
    public static void main(String[] args) {
        drawline();
        System.out.println("Java Lectures");
        drawline();
    }
    public static void drawline()
    {
        for (int i=1; i<=10; i++)
        { System.out.print("*");
        }
        System.out.println();
    }
}
```

Output:



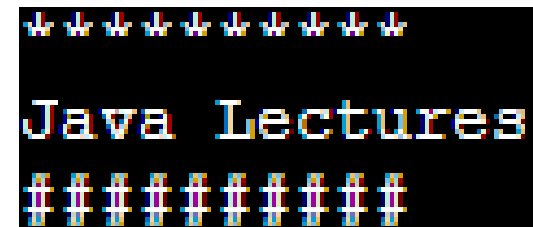
```
*****
Java Lectures
*****
```

Example:

Write a program using methods to write a statement between star lines and # line.

```
public class Main
{
    public static void main(String[] args) {
        drawline('*');
        System.out.println("\nJava Lectures");
        drawline('#');
    }
    public static void drawline(char x)
    {
        for (int i=1; i<=10; i++)
        { System.out.print(x);
        }
    }
}
```

Output:



```
*****
Java Lectures
#####
```

Example:

Write a program using methods to write a statement between two lines have varied length using one function.

```
public class Main
{
    public static void main(String[] args) {
        drawline('*', 10);
        System.out.println("\nJava Lectures");
        drawline('#', 20);
    }
    public static void drawline(char x, int length)
    {
        for (int i=1; i<=length; i++)
            System.out.print(x);
    }
}
```

Output:

```
*****
Java Lectures
#####
```

Example:

Write a program using functions to find the sum of two integer numbers.

```
public class Main
{
    public static void main(String[] args) {
        int x = 20;
        int y = 30;
        System.out.println("sum is"+sum(x,y));
    }
    public static int sum (int i, int j)
    { int result = i + j ;
      return result;
    }
}
```

Output:

sum is 50

Example:

Write a program using functions to find the sum of two integer numbers.

```
public class Main
{
    public static void main(String[] args) {
        int x = 20;
        int y = 30;
        int total = sum (x,y);
        System.out.println(" sum is "+ total);
    }
    public static int sum (int i, int j)
    { int result = i + j ;
      return result;
    }
}
```

Output:

sum is 50

Example:

**Write a program using three functions for:
Sum 3 numbers - Find the average - print the results.**

```
import java.util.Scanner;
public class Main
{
    public static void main(String[] args) {
        Scanner input=new Scanner (System.in);
        System.out.println("Enter the three numbers:");
        int number1 =input.nextInt();
        int number2 =input.nextInt();
        int number3 =input.nextInt();
        int total = sum (number1,number2,number3);
        double avg= average (number1,number2,number3);
        display(total,avg);
    }
    public static int sum (int n1, int n2, int n3)
    { return n1+n2+n3;}
    public static double average (int i, int j, int k)
    { return sum (i,j,k)/3f;}
    public static void display (int t , double a)
    { System.out.println(" The sum is "+ t);
      System.out.println(" The average is "+ a);}
}
```

Output:

```
20
30
The sum is 60
The average is 20.0
```

Scope of Variables:

- Scope is the context within a program in which a variable is valid and can be used.
- Variables: Local and Global.

Local variable:

- Declared within a function (or block)
- Can be accessible only within the function of from declaration to the end of the block.

Global variable:

- Declared outside of every function definition.
- Example: `static int x;` (before main function)

Local Variable:

```
public class Main
{
    public static void main(String[] args) {
        int x =100;
    }
    public static void display()
    {
        System.out.println(x);
    }
}
```

```
Main.java:16: error: cannot find symbol
        System.out.println(x);
                           ^
    symbol:   variable x
    location: class Main
1 error
```

```
public class Main
{
    public static void main(String[] args) {
        int x =100;
        {int y =200;
        }
        System.out.println(y);
    }
}
```

```
Main.java:15: error: cannot find symbol
        System.out.println(y);
                           ^
    symbol:   variable y
    location: class Main
1 error
```

Local Variable:

```
public class Main
{
    public static void main(String[] args) {
        int x =100;
        {
            System.out.println ("x = "+x);
        }

    }
}
```

x = 100

Global Variable:

```
public class Main
{ static int x;
  public static void main(String[] args) {
    x = 100;
    System.out.println ("x = "+x);
    display();
  }
  public static void display()
  { System.out.println ("x = "+x);
  }
}
```

```
x = 100
x = 100
```

Method Overloading:

- Defining several methods within a class with the same name. As long as every method has a different signature.

Method Signature:

The signature of a method consists of the following:

- Method name.
- Formal parameter list
- The returned type value is not part of its signature.

Examples:

- `public int Sum(int x, int y)`
- `public int Sum(int x, int y , int z)`
- `public double Sum(double x, double y, double z)`
- `public int Sum(int x, double y)`

Example:

```
public class Main
{ static int x;
    public static void main(String[] args) {
        int x = 20, y = 30, z = 70;
        System.out.println(getlarger(x,y));
        System.out.println(getlarger(x,y,z));
    }
    public static int getlarger (int a , int b)
    { if (a > b)
        return a;
      else return b;
    }
    public static int getlarger (int a , int b , int c)
    { if (a > b && a > c)
        return a;
      else if (b > a && b > c)
        return b;
      else return c;
    }
}
```

30
70

Call by value Vs call by reference

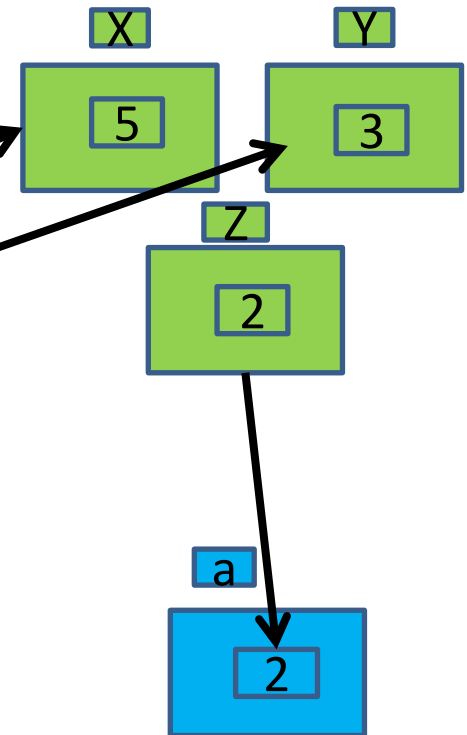
Variables like int, double,... are called by value

Arrays and objects are called by reference (address)

Call by value

```
public static int subtract (int x, int y){  
    int z= x-y;  
    return z;  
}
```

```
public static void main(String[] args) {  
    int a;  
    a= subtract (5,3);  
    System.out.println( a );  
}
```

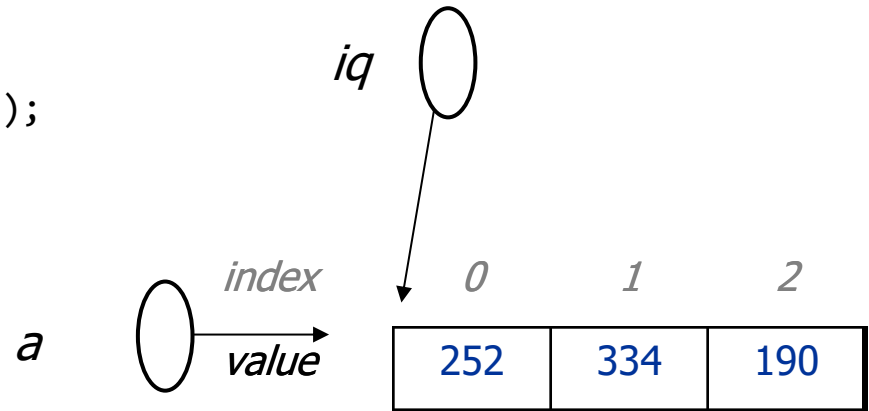


Arrays pass by reference

Arrays are passed as parameters *by reference*.

Changes made in the method are also seen by the caller.

```
public static void main(String[] args) {  
    int[] iq = {126, 167, 95};  
    increase(iq);  
    System.out.println(Arrays.toString(iq));  
}  
  
public static void increase(int[] a) {  
    for (int i = 0; i < a.length; i++) {  
        a[i] = a[i] * 2;  
    }  
}
```



Output:

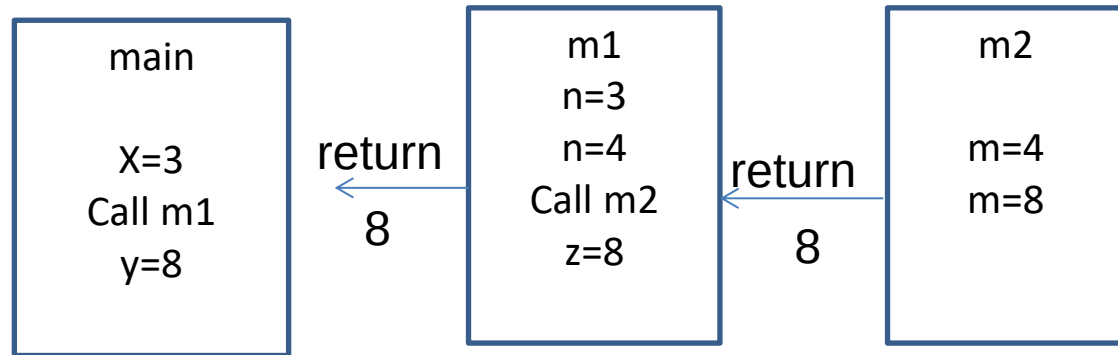
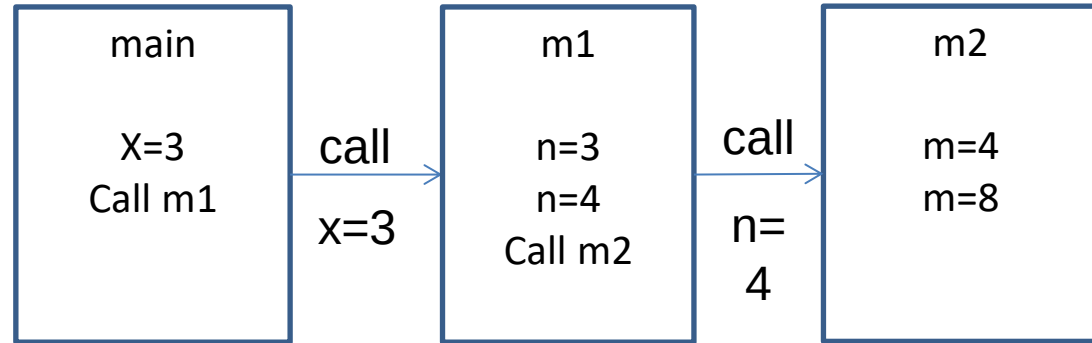
[252, 334, 190]

```
public static void main(String[] args) {
```

```
    int x=3;  
    int y = m1(x);  
    System.out.println( y);  
}
```

```
public static int m1 (int n){  
    n++;  
    int z= m2 (n);  
    return z;  
}
```

```
public static int m2 (int m){  
    m*=2;  
    return m;  
}
```



```
static void m1() {  
    System.out.println("I am inside the m1 method and going to call the m2  
    method. ");  
    m2();  
    System.out.println("The control has returned to the m1 method. ");  
}
```

```
static void m2() {  
    System.out.println("I am inside the m2 method and going to call the m3  
    method");  
    m3();  
    System.out.println("The control has returned to m2");  
}
```

```
static void m3() {  
    System.out.println("I am inside the m3 method.");  
}
```

```
public static void main(String[] args) {  
    m1();  
}
```

I am inside the m1 method and going to call the m2 method.
I am inside the m2 method and going to call the m3 method
I am inside the m3 method.
The control has returned to m2
The control has returned to the m1 method.